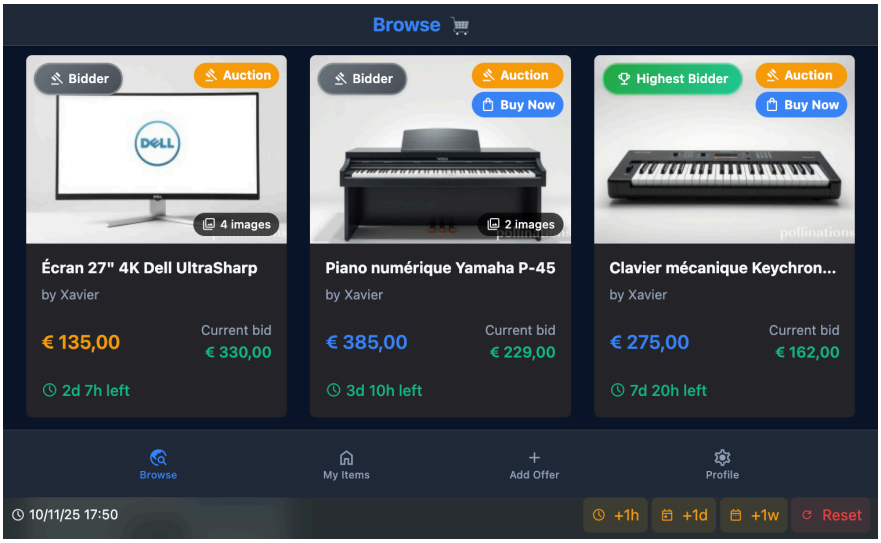
Gestion du temps simulé

Comme une annonce qui contient au moins une offre d'achat ne peut pas être modifiée et qu'une annonce aux enchères se termine au bout d'un certain temps, il est nécessaire de pouvoir tester la clôture des annonces sans attendre que celle-ci se termine effectivement.

Pour ce faire, votre application va utiliser un **temps simulé**, indépendant de l'horloge réelle du système.

Ce mécanisme permet d'avancer dans le temps à volonté tout en maintenant la cohérence des données et donc de tester le comportement de l'application dans le futur.

Dans votre application, cela se matérialisera par une barre permettant de gérer le temps pendant la phase de développement et de test :



Cette barre présente l'heure « officielle » de l'application, qui peut être avancée par incréments d'une heure, d'un jour ou d'une semaine. Dans cet exemple, si l'utilisateur clique sur le bouton « +1 week », l'heure officielle de l'application passera à `2025-11-17 17:50:00` et l'on pourra vérifier que les deux premières annonces sont désormais clôturées et qu'il ne reste que 20 heures pour la troisième annonce.

Principes généraux

- Ne **jamais** utiliser `NOW()` ou `CURRENT_TIMESTAMP` en SQL, ni `date()` directement en PHP pour la logique métier.

- Toujours passer un paramètre `:now` dans les requêtes SQL préparées, dont la valeur provient de `AppTime::get_current_datetime()`.
- Pour tout calcul temporel côté PHP, récupérer l'heure via la classe `AppTime`.
- Pour avancer le temps, utilisez les actions du contrôleur `controller/ControllerTime` fourni. Ces actions sont accessibles via la page d'exemple `/time/index`.

API `AppTime (utils/AppTime.php)`

Méthode	Description
`AppTime::get_current_datetime(): string`	Renvoie l'heure « officielle » de l'application (format `Y-m-d H:i:s`).
`AppTime::add_period(int $amount, string $unit): void`	Ajoute la période fournie à l'offset. `$unit` peut prendre les valeurs `hour`, `day` ou `week`
`AppTime::reset_offset(): void`	Réinitialise l'offset à 0 (retour à l'heure réelle).

Exemples PHP

Voici quelques exemples d'utilisation de l'API `AppTime` dans le code PHP / SQL de votre application :

```
// Avancer l'horloge interne de 2 heures
AppTime::add_period(2, 'hour');

// Obtenir l'heure « courante » de l'application
$now = AppTime::get_current_datetime();

// Exemple d'utilisation de :now dans une requête SQL
$statement = self::execute(
    "SELECT v.*, GREATEST(TIMESTAMPDIFF(SECOND, :now, v.end_at), 0) AS
secs_left
     FROM v_items_status v;",
    ['now' => AppTime::get_current_datetime()]
);

// Autre exemple avec toutes les annonces actives (ouvertes)
$statement = self::execute(
    "SELECT v.*
     FROM v_items_status v
     WHERE v.not_purchased_direct_sale
        OR (v.is_auction
            AND v.end_at > :now
            AND (NOT v.has_buy_now OR NOT v.buy_now_reached));",
```

```
    ['now' => AppTime::get_current_datetime()]
);

// Dernier exemple avec toutes les annonces vendues (clôturées ou achetées)
$statement = self::execute(
    "SELECT v.*
     FROM v_items_status v
     WHERE (v.is_direct_sale AND v.has_bids)
        OR v.buy_now_reached
        OR v.end_at <= :now;",
    ['now' => AppTime::get_current_datetime()]
);
```

Remarques importantes

- L’avancement du temps ne doit être utilisé que dans un contexte de développement ou de test.
- Ne créez aucune annonce ou offre d'achat dans le futur pour ne pas fausser les données (toutes les données doivent être dans le passé : il ne peut par exemple pas y avoir d'annonce ou d'offre d'achat créée dans le futur).
- Toujours réinitialiser l'offset de temps après les tests pour éviter des comportements inattendus.
- L'action `install` du contrôleur `ControllerSetup` permet d'installer des données de test avec des dates spécifiques (pas trop loin dans le passé) pour faciliter les tests liés au temps.
- L'offset est stocké dans la session utilisateur, ce qui signifie qu'il est réinitialisé à chaque nouvelle session.